SUPERSEDED — this is the v2 pack. Do not follow it as instructions.

It describes NEWQuantum_View_Exceptions_Dashboard_fixed.pbix and a 12-step runbook whose model work you have already applied. **The current guide is UPS_QuantumView_v3_Instructions.pdf / V3-RUNBOOK.md.**

Keep this for the reasoning behind the exception classifier, the dedupe direction, the account key derivation and the Shipper Match Key finding — that analysis still holds. But do not re-run model-fixes-v2.pq sections 1 to 9 or model-updates-v2.tmdl. The only part of v2 still outstanding is **section 10, the trace date columns**.

BUILD AND REMEDIATION INSTRUCTIONS · V2

NEWQuantum_View_Exceptions_Dashboard_fixed.pbix

Everything you need to take the attached file from "opens but will not refresh" to production-ready. Twelve Desktop steps, about 55 minutes. Appendix A covers using a .pbit template with a .pbix.

What you have	What it is for
NEWQuantum_View_Exceptions_Dashboard_fixed.pbix	The working file. Report layer repaired, sensitivity label removed so it opens. The model inside is byte-identical to what you sent.
quantum-view-scripts.zip	The four scripts and seven verification queries you paste in Desktop. **Copy the code from these files, never from this PDF** - a PDF turns straight quotes into curly ones and hyphens into dashes, and M rejects both.
This document	The instructions. Also in the repo as NEW-BUILD-FIXES.md, WRITEBACK-AND-MAILMERGE.md and PBIT-GUIDE.md.

Before you open anything: do not click Apply on the pending query change.

Your file carries an unapplied Power Query edit that calls `fnClassifyException`. That function does not exist anywhere in the model - not as a query, a function or an expression. I checked the applied model and all 29 queries. The data you see today is cached from a refresh that ran before the edit was made, so the file looks fine until the moment you press **Close & Apply** or **Refresh** - at which point QV_Enriched fails and takes QV_Output, QV_Manifest, Resolutions Table and every visual in the report down with it. Step 3 replaces that edit with a corrected version. Discard what is sitting in the editor; do not apply it.

The sensitivity label has been removed. Put it back before the file leaves your machine.

SecurityBindings is a tamper binding computed over the whole package, so any edit made outside Desktop invalidates it and Desktop then reports the file as corrupted. The label is Method=Standard, ContentBits=0 - a classification marking, not rights-management encryption - so removing it costs nothing except the marking itself. **The attached file is an unclassified copy.** Step 11 puts the label back, and saving from Desktop regenerates a valid binding.

At a glance

What was wrong	Severity	Fixed by
The next refresh fails - an unapplied edit calls a function that does not exist	Blocking	Step 3

What was wrong	Severity	Fixed by
Shipper Match Key cannot carry a relationship - 23 accounts, 17 unique keys	**Blocking**	Steps 3, 4, 5
The resolution queue shows each exception's *oldest* state, not its newest	**Silent**	Step 3
Dim_ExceptionReason loses 16% of the queue to the blank member	**Silent**	Step 3
Two Exception Type columns, one trailing space apart, disagreeing on 28,286 rows	**Silent**	Steps 2, 3
The measure sweep did not survive the round trip	**Accuracy**	Steps 1, 2, 5
Branch_Contacts loads 500 rows of pure nulls	**Blocks Flow 2**	Step 8
No per-source freshness anywhere in the report	**Requested**	Steps 8, 8b

"Silent" means the dashboard shows a confident number that is wrong, with no error anywhere - the failure mode worth fixing first, because nothing tells you it is happening.

SECTION 1

What was wrong

Both new dimensions had a real defect. Two worse problems turned out to be sitting underneath them, and neither had anything to do with the dimensions.

1. The next refresh fails

Covered in the warning on page 1. **tools/model-fixes-v2.pq** section 1 is the missing function, written to be worth having rather than just to unblock you - it is priority-ordered and it never returns null.

2. Shipper Match Key can never carry a relationship

This is the error you were hitting. `Dim_Account[Shipper Match Key]` is UPPER(TRIM(Account Name)), and five of your accounts are all called **EDWARD JONES MAIL SERVICES** - 75V664, 8V7494, 8V7507, 9V2538 and 9V2549. Two more pairs collide the same way. **23 accounts collapse to 17 keys.** The "one" side of a many-to-one has to be unique, so Power BI refuses to build the relationship. Nothing you do on the fact side changes that.

It would not have worked even if it were unique, because Shipper Name in the shipment feed is not the account name:

Table	Shipper-name match	Tracking-derived Account Key
QV_Output	21.72%	**99.13%**
QV_Manifest	48.42%	**99.18%**
Resolutions Table	31.19%	**98.83%**

The misses are systematic, not typos: `EDWARD JONES/GATEWAY CDI` (25,107 rows) against the workbook's `EDWARD JONES/GATEWAY CDI SRM`; `EDWARD JONES/ CVS` (5,896) against `EDWARD JONES/ICVS`; and roughly 40,000 rows where Shipper Name is an individual's name, not a business at all.

The fix is the account already embedded in the tracking number. UPS 1Z format is 1Z + 6-character shipper account + 2-character service + 8-character package id, so characters 3 to 8 are the account:

```
1Z 2093RE 03 01360983   ->  2093RE   EDWARD JONES/Brand Addition
1Z 8E19V7 02 95095857   ->  8E19V7   EDWARD JONES/ICVS
1Z XH8186 03 39818208   ->  XH8186   Nova EDWARD JONES DISTRIBUTION
```

You already derive it exactly this way on Trace_Active.

A25T52 is still missing, and it is the entire remainder

Table	Unmatched rows	All of them
QV_Output	404	A25T52
QV_Manifest	756	A25T52
Resolutions Table	102	A25T52

Add it and coverage is 100% on every table. The prepared row is in **UPS_Acct_Info_.xlsx**; your live workbook is `\\nfpgshare-1...\Report Data\UPS Acct Info .xlsx` and **the table range has to be extended to A1:L25**

or the refresh will not see the new row.

3. Dim_ExceptionReason silently drops 16% of the queue

The dimension is fine - 11 rows, all 11 keys match. The problem is what does not reach it. **1,406 of 8,724 Resolutions Table rows have a null Exception Type**, so they land on the blank member: invisible in the slicer, absent from the category chart, uncountable. One exception in six.

They are not junk. 1,257 of them are one business concept:

Description	Rows
WE TRIED TO DELIVER TO THE BUSINESS, BUT IT WAS CLOSED...	916
WE TRIED TO DELIVER TO THE BUSINESS AGAIN, BUT IT WAS CLOSED...	127
THE RECEIVING BUSINESS WAS CLOSED. WE'LL ATTEMPT...	103
THE RECEIVING BUSINESS WAS CLOSED AT THE TIME OF THE FINAL ATTEMPT	54
THE RECEIVER WAS NOT AVAILABLE FOR DELIVERY...	37
the rest of the attempt / miss family	20

The fix has two parts. The **guarantee**: the classifier never returns null, unmatched descriptions fall to "Other Exception", and the dimension has a row for it. No judgement calls, nothing can vanish. The **taxonomy**: 18 new keyword rules, all appended below the existing 29 so they can only catch what nothing else caught. Verified - **zero previously-classified rows change.** Unclassified goes 1,394 to 64 on the queue (16.0% to 0.7%) and 2,445 to 207 on QV_Output. The headline new type is **Delivery Attempt**: 1,273 rows, 15% of your queue, and genuinely actionable. Delete any rule you disagree with - the fallback catches the rest.

And the fact's Exception Category was never a category

`Resolutions Table[Exception Category]` and `QV_Output[Exception Category]` disagree with the dimension on **36.5% of matched rows**. Not a data problem - a naming one. Inside QV_Enriched the column is literally defined as an alias of the type:

```
#"Added Exception Category Alias" =
    Table.AddColumn( ..., "Exception Category",
                     each [Exception Type Detail], type text ),
```

So the fact says `Access Point Hold` (2,347 rows) where the dimension says `Access Point`, and `Wrong Recipient` (213) where the dimension says `Delivery Issue`. Two columns with the same name at two different grains, indistinguishable on screen. The dimension is authoritative; the classifier now returns both the leaf type and the real rollup from one shared map.

Two Exception Type columns, one character apart

Column	Source	How it picks
`Exception Type`	DAX calculated column	MAXX - the **alphabetically last** match
`Exception Type` *(trailing space)*	Power Query	the **first** rule in table order

They disagree on **28,286 of 46,594 rows**. The DAX one is wrong by construction - MAXX knows nothing about priority, so "MISSING SUITE NUMBER" scores Package Issue over Missing Suite Number, and "ACCESS POINT ... HOLD" scores Schedule Change over Access Point Hold. The Power Query one is right, but only by accident: your Keyword Table happens to be authored in priority order and nothing recorded that. Delete the DAX column; the keyword table gets an explicit Priority column so re-sorting the sheet cannot silently reclassify thousands of rows.

4. The resolution queue is showing each exception's oldest state

This has nothing to do with the dimensions and it is the most consequential thing I found.

Resolutions Table sorts Date modified descending and then takes Table.Distinct on Exception Key, intending to keep the newest sighting. **It does the opposite.** Table.Distinct does not honour an unbuffered Table.Sort - without Table.Buffer, Power Query streams and keeps whichever row it meets first, which is source order.

Measured on your data, across the 4,075 exception keys that appear in more than one export:

Outcome	Keys
Kept the **newest** row	**0**
Kept the **oldest** row	**4,075**

So for 47% of the queue, Status, Exception Resolution, Scheduled Delivery and everything else are the *first* thing UPS ever said about that exception, and every update since has been discarded. **QV_Manifest has the identical construction** in its "Latest per Shipment" step, so it has the identical defect. The fix is Table.Buffer around the sorted table - one function call, both queries.

Ageing has to move at the same time. Age Days currently runs off the surviving row's Date modified, which by luck is the first sighting - so it is accidentally right today and would silently invert the moment the buffer fix lands. Section 8 of the .pq computes First Seen explicitly off the full table before the dedupe, so ageing is right on purpose: a recurring exception keeps ageing instead of resetting to zero every time it reappears. **1,110 rows change Age Bucket**, mostly out of 0-1 into 8+, which is the honest picture.

5. The measure sweep did not survive the round trip

	Now	Should be
`Exception Rate`	events / shipments - mixed grain	distinct affected / distinct shipments **9.31% to 8.60%**
`Successful Shipments`	inherits the same mixed grain	same correction
`Aged 8 Plus Days`	counts resolved cases too	Tracker Status <> "Resolved"
`Avg Age (Days)`	plain AVERAGE, counts resolved	same
3 duplicate measures	byte-identical copies	delete - verified unused
Auto Date/Time	on - 12 hidden LocalDateTable_* plus a template	off
`Dim Date`	does not exist	the conformed calendar
`QV_Manifest`	orphaned, no relationships at all	related to Dim_Account and Dim Date

Until QV_Manifest has a relationship, Total Shipments is a flat 92,630 under every filter, Exception Rate is wrong on any sliced view, and the vendor ranking chart on OVERVIEW shows the same total on every bar.

6. Smaller things, all fixed in the scripts

- **Branch_Contacts loaded 500 rows of pure nulls.** The query promotes row 1 of the "branch email" sheet to headers, but row 1 is not the header row - two of the column names it produced were "Source" and a literal file path, so a provenance block sits above the real header. It loaded without erroring, which is why it survived. Rebuilt in tools/contacts-and-freshness.pq.
- **Review Status writes "Escalate " with a trailing space** while the measure Escalated Traces matches "Escalate" with none. DAX does not trim. It reads 67 today, so the space is being lost somewhere between Power Query and the model - but that is luck, not design.
- **Trace dates are still text.** Every date column on Current_Open_Trace and Trace_Active loads as text, so no calendar can reach the trace side - no date slicer, no trend, no SLA maths on the page whose entire job is SLA.
- **Account Label has a double space on one row** - XH8186 - Nova EDWARD JONES DISTRIBUTION, because the workbook name has a leading space and the label concatenates it raw. Cosmetic, but it is the slicer's display text.
- **Resolution Status is starting to flow** - 1,039 rows now carry Open, up from zero. Root Cause, Next Action, Notes, Last Updated and Closed Date are still 100% empty across all 8,724 rows. That is the write-back gap; see Section 5.

SECTION 2

What is already done in the file

Report layer only. **DataModel is untouched and verifiably byte-identical** to your upload - it is a compressed Analysis Services backup and cannot be edited from outside Power BI Desktop, which is why all the model work below is scripts rather than something I could just do.

Five visuals repointed at the dimension

Each keeps its own header text, column width and sort, so nothing moves on screen.

Page	Visual	Was	Now
OVERVIEW	Exceptions by Category	QV_Output[Exception Category]	Dim_ExceptionReason[Exception Category]
OVERVIEW	exception detail table	QV_Output[Exception Type]	Dim_ExceptionReason[Exception Type]
RESOLUTION QUEUE	queue table	Resolutions Table[Exception Category]	Dim_ExceptionReason[Exception Category]
RESOLUTION QUEUE	exception slicer	Resolutions Table[Exception Type]	Dim_ExceptionReason, Category > Type - the same two-level hierarchy as OVERVIEW
Exception Details	detail table	QV_Output[Exception Category]	Dim_ExceptionReason[Exception Category]

Three things cleared that were baked into the file

— A saved slicer selection on RESOLUTION QUEUE.

— Another on the hidden TEST-Acct Slicer page.

— A stuck drill-through value on Exception Details - that page was opening pre-filtered to one hard-coded exception (1Z0708F90100725981 | WE TRIED TO DELIVER...).

The OVERVIEW filter rail is back on the grid

It had drifted to an 80px slot pitch with 120/126/128px widths, while RESOLUTION QUEUE and ACTIVE TRACES sat on the spec's 68px pitch at a uniform 126. All three rails now line up exactly:

```
x 36    y 196 / 264 / 332 / 400 / 468 / 536    126 x 56
```

Also squared up the five page-title textboxes and five FILTERS labels, and took the trailing space out of the ACTIVE TRACES slicer header ("Vendor Acct # ").

> **Verified after the edits**
>
> 180 field references resolve · 0 overlaps · nothing off-canvas · all 106 report JSON parts parse · DataModel SHA-256 identical to your upload.

SECTION 3

The runbook

Twelve steps, about 55 minutes. Do them in order - each one gates the next. Keep a copy of the file you were sent before you start; every step is reversible, but a known-good starting point costs nothing.

Copy code from the script files, not from this PDF

A PDF renders straight quotes as curly quotes and hyphens as en-dashes. M and DAX both reject those, and the error message will not tell you that is what happened. Open the .pq and .tmdl files from quantum-view-scripts.zip in Notepad++, VS Code or even Notepad, and copy from there.

Step 1 · Turn off Auto Date/Time — 1 min

`File > Options and settings > Options > Current File > Data Load` - untick **Auto date/time**.

This has to happen before Step 5, or Power BI keeps generating a hidden calendar behind every date column and you end up with two competing date tables. Your file currently carries twelve of them plus a template.

✓ The little date hierarchies disappear from under each date column in the Fields pane.

Step 2 · Delete four fields — 2 min

Fields pane, right-click, *Delete from model*:

```
QV_Output[Exception Type]              <- the DAX column. MAXX-based,
                                          wrong by construction
'Resolutions Table'[Exceptions Total]  <- duplicate of QV_Output[Total Exceptions]
QV_Output[Open Exceptions]             <- duplicate of Resolutions[Open Ex]
QV_Output[Exceptions Resolved]         <- duplicate of Resolutions[Resolved Exceptions]
```

All four verified unused by any visual. **The first one has to go before Step 3**, or the new Exception Type column collides with it.

✓ No visual shows an error afterwards. If one does, undo with Ctrl+Z - you deleted the wrong copy.

Step 3 · Power Query - the main fixes — 20 min

`Home > Transform data`. Work through **tools/model-fixes-v2.pq** in order. Each section says what it replaces and why.

Section	Query	What it does
1	fnClassifyException *(new)*	The missing function. Priority-ordered, never returns null. Both lookups are built once in the enclosing let and the function closes over them, so it does not re-scan for every one of 46,000 rows.
2	Exception Reason Map *(new)*	One place the taxonomy lives. The classifier reads it for the rollup; Dim_ExceptionReason is it.
3	Keyword Table	Adds an explicit Priority column, plus 18 new rules appended below the existing 29.
4	Dim_ExceptionReason	Becomes the map, straight through, including the Other Exception member. 11 rows to 14.
5	QV_Enriched	**Your pending edit, with three corrections.** Paste this instead of applying what is in the editor.
6	QV_Output	Drops the duplicate inline classifier, adds Account Key.
7	Dim_Account	Drops Shipper Match Key, trims Account Label.
8	Resolutions Table	Table.Buffer on the dedupe, First Seen for ageing. **Read the QV_Manifest note at the end of this section** - the same two-line fix applies there.
9	Current_Open_Trace	One character: "Escalate " becomes "Escalate".

Then **Close & Apply**.

- ✓ **The refresh completes.** That alone is the section 1 fix landing.
- ✓ Dim_ExceptionReason has **14 rows**, including Other Exception.
- ✓ Dim_Account has no Shipper Match Key, and every fact table has an Account Key.

Step 4 · Add A25T52 to the workbook

5 min

Add the row from **UPS_Acct_Info_.xlsx** to `\\nfpgshare-1...\Report Data\UPS Acct Info .xlsx`, **extend the table range to A1:L25**, then refresh.

The sheet is backed by a named table (tbl_Account_List) and Power Query sources from it - appending a row without extending the range means the refresh never sees it.

Field	Value
Account #	A25T52
Account Name	EDWARD JONES CANADA / CBIZ NS
Address	4020A SLADEVIEW CRES, MISSISSAUGA ON L5L 6B1
City / St	MISSISSAUGA / ON
Status	Active
Internal Contact Name	UNVERIFIED - derived from shipment data

The name is inferred from shipment data, not from UPS billing - that is flagged in the Internal Contact Name cell on purpose so nobody mistakes it for verified. The evidence is unambiguously Canadian: ship-to provinces ON (574), BC (63), AB (36), NS (12); the dominant shipper is CBIZ NS-EDWARD JONES at 4020A Sladeview Cres, Mississauga. Replace both fields once someone confirms what the business actually calls this account - the slicer label is built from Account Name, so it follows automatically. Zip Code is deliberately blank: that column is numeric with an 00000 format, and putting a Canadian postal code in it would make the column mixed-type and throw on refresh.

✓ Dim_Account shows 24 rows and no tracking number fails to match.

Step 5 · Apply the model script

5 min

`View > TMDL view`, new script, paste **tools/model-updates-v2.tmdl**, read it, **Apply**.

Two things to do to the script before you apply it

Stop before section 9. It relates the calendar to the trace tables and cannot work until Step 6 types those columns. Delete section 9 for now, or let it fail and re-run it after Step 6 - everything above it still lands.

Skip section 1 if you took the Power Query route in Step 3. It is the same two account-key columns done twice.

Every measure keeps its name, so **no visual needs rebinding** - only expressions change.

✓ **Exception Rate reads 8.60%**, not 9.31%.

✓ Dim Date appears in the Fields pane and **starts in 2026**. If it starts in 1900, stop and read Step 6 - a text date has been converted naively somewhere.

✓ **The vendor ranking chart on OVERVIEW is no longer flat.** That is the account relationship landing.

Step 6 · Type the trace date columns

10 min

`Home > Transform data`, select **Current_Open_Trace**, `Advanced Editor`. Paste the block from **tools/model-fixes-v2.pq section 10**, placed **after** the Trace_Notes merge step. Then repeat for **Trace_Active**.

Two things in that code are doing real work:

— **"Not Avail." and 1/3/1900 become null, not dates.** 28 of 95 Follow-up values are 1/3/1900 - an Excel epoch artefact from a blank cell that got date-formatted. Convert those naively and Dim Date stretches back to 1900, roughly 46,000 rows, and every date axis becomes unreadable.

— **Culture = "en-US" is pinned explicitly.** "7/8/2026" is 8 July under en-US and 7 August under en-GB. Without the culture pinned, a Service refresh can silently disagree with Desktop.

Replace `PREVIOUS_STEP` in that block with the real name of the step above it - on Current_Open_Trace that is `#"Expanded Trace_Notes"`. Then **Close & Apply**.

✓ Manifest Date on Current_Open_Trace shows a **calendar icon**, not *abc*.

✓ Follow-up has **28 blanks**. Dim Date still starts in 2026.

Step 7 · Relate the calendar to the trace side

2 min

Re-run **section 9** of model-updates-v2.tmdl, the part you skipped in Step 5. Or do it by hand in Model view:

From (many)	To (one)	Active
Current_Open_Trace[Manifest Date]	'Dim Date'[Date]	**yes**
Current_Open_Trace[Last Reviewed]	'Dim Date'[Date]	no

✓ Model view shows Dim Date connected to three fact tables.

Step 8 · Branch contacts and refresh stamps

12 min

`Home > Transform data` again, then **tools/contacts-and-freshness.pq**:

Section	What to do
1 - discovery *(one-off)*	New blank query, paste, look at the four columns it returns. Note the item name, its Kind, and which row holds "Email". **Then delete the query.** I cannot see your workbook from here, so this is the step that replaces guessing.
2 - Branch_Contacts	Set ItemName, ItemKind and KeyColumn from what section 1 showed you, then replace the whole existing query. **If section 1 shows a Kind = "Table" item, use that instead** - a named Excel table carries its own header and cannot drift, and the query collapses to the six lines at the bottom of the section.
3 - Refresh Status *(new)*	New blank query, rename it **exactly** Refresh Status.

Close & Apply. Then `View > TMDL view`, paste **tools/contacts-and-freshness.tmdl**, **Apply**. That adds the roster relationship and nine measures. It touches nothing that already exists, so nothing can regress.

✓ Branch_Contacts has real rows with real addresses - **not 500, and not 0**.

✓ Refresh Status has **5 rows** and every Modified is populated. A null means that source's folder or filename does not match what section 3 expects - fix the pattern, not the data.

Step 8b · Point the title bars at the right source

3 min

Five cards, one per page, at `x 868 · y 27 · 372 x 38`. Each currently shows `Max(QV_Output[Date modified])`. For each one: click the card, drag the measure below into the **Data** well, remove the old field, and rename the field to `DATA AS OF` so the label does not change.

Page	Measure
OVERVIEW	[Data As Of]
RESOLUTION QUEUE	[Data As Of]
Exception Details	[Data As Of]
ACTIVE TRACES	**[Trace Data As Of]**
Trace Details	**[Trace Data As Of]**

The trace pages get the tracing workbook's stamp because that is what actually feeds them - right now they show the QV export's date, which has nothing to do with what is on the page. What you will see:

```
8/5 2:32 PM
8/5 2:32 PM   .   1 SOURCE STALE
NO FEED FILES FOUND
```

Why this is not already done in the file

Both measures live on Refresh Status, a table that does not exist until Step 8 runs. Binding a visual to a missing table is the exact risk profile that produced the unopenable files earlier in this build, and I am not spending your openability on a three-minute step. Same reason the Manifest Date slicer shipped bound to Resolutions Table[Manifest Date] rather than Dim Date[Date].

Step 9 · Two follow-ups — 2 min

- **Repoint the date slicers.** On OVERVIEW and RESOLUTION QUEUE the Manifest Date slicer is bound to Resolutions Table[Manifest Date] - chosen so it worked before Dim Date existed. Swap it for 'Dim Date'[Date]. It then also filters QV_Manifest, which is what makes Total Shipments and Exception Rate date-responsive.
- **Add a date slicer to ACTIVE TRACES.** Possible for the first time. Drag 'Dim Date'[Date] into the rail, then set Format > General > Properties > Position to `X 36 · Y 604 · W 126 · H 56` - rail slot 7, matching the other two pages exactly.
- **Mark the date table** if it is not already: Dim Date > Table tools > Mark as date table > column Date.

Step 10 · Verify — 5 min

`View > DAX query view`. The seven files in **tools/checks/** are read-only - they change nothing.

Query	Confirms
7-classification-coverage.dax	**Unclassified = 0**, and no fact type without a dimension row
8-dedupe-recency.dax	**Stale rows = 0** - the queue is showing current state
9-sources-and-contacts.dax	**Branch contacts loaded > 0**, 5 sources current, and the 23 traces the roster rescues

Query	Confirms
4-kpi-reconciliation.dax	Exception Rate 8.60%
5-status-vocabulary.dax	the exact strings your status measures match on
3-age-days-anchor.dax	ageing measures case age, not run recency
6-trace-notes-join.dax	notes actually land

Then click through each page. Every KPI should move when you change the **Vendor Acct #** slicer, and ACTIVE TRACES should read **95 / 67 / 28 / 27 / 86**.

Step 11 · Re-apply the sensitivity label

1 min

`Sensitivity > Internal Use Only · Standard`. Saving from Desktop regenerates a valid tamper binding.

Do this before the file leaves your machine.

Step 12 · Publish

15 min

- Publish to a **dedicated workspace**, not My Workspace.
- Point Power Query credentials at a **service account**, not your login - otherwise everything breaks the day you are on PTO or change roles.
- **Gateway:** needed for the UNC path; **not** needed if the landing folder moves to SharePoint and you switch to SharePoint.Files.
- Scheduled refresh **before** the 08:00 SOP window.
- Publish as an **App** to the ops team rather than sharing the raw report.
- Hide or delete the **TEST-Acct Slicer** page - currently hidden, not deleted.

SECTION 4

Write-back and mail merge

Spec pages 12 and 13. The largest remaining piece, and the only part of the build that lives outside Power BI. Full detail is in WRITEBACK-AND-MAILMERGE.md - this is the shape of it and the two decisions that are already made.

What I cannot build from here, and why

The write-back panel is a **Power Apps visual** and the send button is a **Power Automate visual**. Neither can be created by editing a .pbix from outside Desktop - they carry an app ID and a connection that are minted against your tenant when you add them. Hand-writing visual JSON for a type Desktop did not author is also exactly what produced the unopenable files earlier in this build. So the placeholder panel on RESOLUTION QUEUE stays where it is until you run this, and the document is a complete build rather than a partial one.

The read side is already finished

Piece	State
Stable join key Exception Key	Done - drill-through keys on it
tbl_Resolution_Input merged onto Resolutions Table	Done - left outer on Exception Key
Resolution Status to Tracker Status	Done - blank means "New / Needs Review"; 1,039 rows carry Open
Root Cause, Next Action, Notes, Last Updated, Closed Date	Columns exist, **0% populated**
Selected Root Cause / Next Action / Notes measures	Built, on the RESOLUTION QUEUE detail panel
Notes panel placeholder	650, 442, 606 x 254

Every column the write-back needs to land in already exists and is already on screen. What is missing is the thing that writes to them.

Decision 1 - the SharePoint List is the system of record

Today tbl_Resolution_Input reads ResolutionTracker.xlsx, which cannot hold a persistent timestamp: an Excel NOW() recalculates every time the file opens, so it can never record when a change actually happened. A List's Modified field is written once, at save, and never drifts. That is the audit trail. Repointing the query is one change; everything downstream is unaffected.

SharePoint.Tables refreshes in the cloud with no on-premises gateway, which is the single biggest operational advantage available here - all the UNC-path queries need one.

Decision 2 - the exceptions recipient comes from the List's Owner

Fixing Branch_Contacts does **not** give the exceptions mail merge a recipient, and it is worth being clear about why before you plan around it. **The exceptions queue carries no FA identity to join a roster to.** QV_Output[Ship To Name] is EDWARD JONES on 35,126 of 46,594 rows and a client's name on most of the rest; QV_Manifest[Ship To Attention] is EDWARD JONES or blank on 40,770. Neither identifies a person, so a working roster has nothing to attach to.

So Flow 2 takes its recipient from the List's **Owner** person column, populated by the Power Apps panel from User() on save. Every exception in the List has an owner because somebody touched it to get it there. That is the design, not a fallback: it needs no roster, it works from day one, and it addresses the person who took the case rather than whoever the shipment happened to be addressed to.

The roster's job is the trace side. Current_Open_Trace[FA #] is populated on all 95 rows and no FA # maps to two different addresses. CONTACT INFO covers 72 of 95; the roster fills the other 23. [Selected Branch Email] resolves the case contact first and falls back to the roster, and [Unreachable Traces] tells you how many you still cannot reach - that number should trend to zero and is worth a KPI slot.

Build order, and what gates what

	Step	Gated on
1	Confirm the tenant permits Flows and Lists	nothing - do it first
2	Create the List, seed 5-10 rows keyed to real Exception Keys	-
3	Repoint tbl_Resolution_Input at the List	List exists
4	Embed the Power Apps panel, test the round trip	List + repointed query
5	Flow 1 - the append-only audit log	round trip proven
6	Confirm the List Owner is populating	Power Apps panel live
7	Flow 2 - mail merge, behind the preview gate	Flow 1 + a recipient source
8	Flow 3 - the scheduled digest	Flow 2 proven on 2-3 rows

Check step 1 before you build anything else - some Edward Jones tenants restrict flow creation, and it decides whether this is two days or two weeks. If it is blocked, the layout and the pipeline are unchanged and only the automation swaps: an Office Script replaces Flow 1, Outlook's native mail merge replaces Flow 2. Every timestamp stays persistent either way.

The preview gate is non-negotiable. The Send button opens a preview of the merged batch and nothing dispatches until the analyst confirms count and template. That is the guardrail against a 148-email misfire.

Run every Power Apps and Power Automate connection on a service account. Otherwise the write-back and every flow break the day you are on PTO or change roles - spec page 15, and the single most common way builds like this die.

APPENDIX A

Using a .pbit with a .pbix

> **The short answer**
>
> **You cannot apply a .pbit to an existing .pbix.** There is no import-template command in Power BI Desktop. A template is a starting point, not a patch - opening one always produces a brand-new report, never a modification of a file you already have.

So the question really resolves into one of four things, and they have different answers.

What you actually want	How it works
Start a new report from a template	**File > Open** the .pbit. Supported, one step
Move **model** objects into an existing .pbix	TMDL view, copy the script across. Supported
Move **report** pages into an existing .pbix	Copy and paste visuals page by page. Works, but fiddly
Ship the build without shipping the data	This is what .pbit is *for*

What a .pbit actually is

The same OPC zip package as a .pbix, minus the data.

Carries	Does not carry
Every Power Query query and parameter	Any data rows at all
The full model schema - tables, columns, measures, relationships, hierarchies, RLS roles, format strings	Cached visual results
The entire report layout and theme	Your credentials

Your .pbix is 7.03 MB. The equivalent .pbit will be a few hundred KB, because 7 MB of it is compressed data.

Creating one from your file

```
File  >  Export  >  Power BI template  >  type a description  >  Save
```

The description is worth writing properly - whoever opens the template sees it in the prompt dialog, and it is the only context they get.

If Export is greyed out, you have unapplied query changes. Apply them first. Your current file is in exactly that state - see the warning on page 1.

Opening one

File > Open the .pbit, or double-click it. Desktop then:

1. **Prompts for every parameter** marked Required, pre-filled with the value that was current when the template was exported.
2. **Runs a full refresh** against every source.
3. Builds the model and the report.
4. Hands you an **Untitled** report - **Save As** to make it a .pbix.

> **The second step is the one that bites**
>
> A template open is a full refresh, so every source has to be reachable *and* credentialed on that machine, right then. On this build that means the \\nfpgshare-1... UNC share - open the template off the network and it fails on the first query. One more reason to move the ingestion to SharePoint.Files before you distribute anything.

Moving model objects into an existing .pbix

This is the supported "merge" path, and it is what the .tmdl files in tools/ already are.

1. Open the template - it becomes an untitled .pbix.
2. **View > TMDL view.**
3. In the TMDL explorer, select the tables, measures or relationships you want and script them out.
4. Copy that script.
5. Open your real .pbix, **View > TMDL view**, paste, read it, **Apply**.

Two things to know before you do it:

— **createOrReplace replaces the whole object.** Scripting out a table brings its partition, its columns and its measures with it - if the target already has that table with different queries behind it, you will overwrite them. Script the smallest thing that does the job, which is usually just the measures.

— **Relationships need both endpoints to exist first.** Same ordering rule as the scripts in tools/ - it is why model-updates-v2.tmdl section 9 has to wait for the trace date columns.

Moving report pages into an existing .pbix

No import command. What works:

1. Open both files in Desktop, two windows.
2. On the source page, click the canvas, **Ctrl+A**, then **Ctrl+C**.
3. In the target file, add a page and **Ctrl+V**.

Fields rebind **by name**. Anything the target model does not have comes across as a broken visual with a "can't find field" error - recoverable, but it means the model has to go first. Background shapes and locked objects paste too, but **page-level settings do not**: canvas size, page background, drill-through configuration and page filters all reset and have to be redone by hand.

For anything more than a page or two it is cheaper to start from the template and repoint its queries than to paste pages into an existing file.

Why .pbit is worth using on this build specifically

Five concrete reasons, in rough order of payoff.

1. Parameters, done properly

The SharePoint-versus-folder switch is exactly what a template prompt is for. Define SourceMode, LandingFolder and SharePointSite as **real Power Query parameters** (Home > Manage Parameters, not just a let step) and mark them Required. Whoever opens the template gets asked for them before a single query runs, so the same template serves test and production without anyone editing M.

2. No data leaves the building

A .pbit contains zero rows. Your .pbix carries 92,630 manifest rows including client names and ship-to addresses. For an Internal Use Only build, that is the difference between handing someone the design and handing them the data.

3. The sensitivity-label problem disappears

SecurityBindings is a tamper binding computed over the whole package, which is why every edited copy in this project has needed the label stripped and re-applied. Desktop rebuilds a template from scratch on open, so there is nothing to invalidate. If this build has to go to another person or another tenant, .pbit is the clean vehicle.

4. It is version-controllable in a way a .pbix is not

In a .pbix the model lives in `DataModel` - a compressed Analysis Services backup, opaque to everything except Power BI. In a .pbit the same model is `DataModelSchema`: plain JSON. Commit a .pbit at each milestone and you get a diff you can actually read. Commit a .pbix and you get "binary files differ".

5. It would let me edit the model directly

This is the one worth thinking about. Every model change in this project has had to be written as a script for you to paste in Desktop, because DataModel cannot be edited from outside Power BI - that constraint is the reason model-updates-v2.tmdl and contacts-and-freshness.tmdl exist in the form they do. DataModelSchema in a .pbit is editable JSON. **Export a .pbit, send it to me, and I can make measure, column and relationship changes directly**, hand it back, and your Desktop work drops to: open it, enter the parameters, Save As.

The catch is worth stating plainly: a template open is a full refresh, so you would need every source reachable at that moment, and the report layer still has the same copy-paste constraints. But for model work specifically it is a materially better loop than what we have been doing.

A rollout pattern that works

Working file	the .pbix on your machine. Never shared directly
Versioned artifact	export a .pbit at each milestone, commit that
Distribution	publish the App from the Service. Nobody downloads either file
Handover / DR	the .pbit plus the repo reconstructs the whole build from nothing

The last row is the one people skip and then regret. Right now, if your machine died tomorrow, rebuilding this dashboard means re-deriving several weeks of decisions. A .pbit in source control means it means opening a file.

Gotchas, collected

Symptom	Cause
Export greyed out	apply pending query changes first
Template open fails immediately	a source is unreachable or uncredentialed - it is a full refresh, not a lazy load
Parameters show the wrong defaults	they bake in whatever was current at export; re-export after changing them
A broken query is still broken	.pbit carries queries verbatim, errors included. It is not a repair mechanism
Pasted visuals show "can't find field"	the model has to be in place before the report layer
Page filters and drill-through vanish on paste	page-level settings do not travel with copied visuals
RLS roles carry, role *members* do not	membership is assigned in the Service, per workspace

APPENDIX B

File manifest

Everything in quantum-view-scripts.zip, and what each piece is for.

Scripts you paste in Desktop

File	Used in	What it is
model-fixes-v2.pq	Steps 3, 6	Nine Power Query sections plus the trace date block. Sections 1-3 are required - without them the next refresh fails.
model-updates-v2.tmdl	Steps 5, 7	Account keys, status-aware ageing, the rate-grain correction, Selected Exception Category, six relationships and the Dim Date calendar.
contacts-and-freshness.pq	Step 8	Roster discovery, the Branch_Contacts rebuild, and the Refresh Status source scan.
contacts-and-freshness.tmdl	Step 8	The roster relationship and nine contact / freshness measures.

Verification queries - read-only, they change nothing

File	Confirms
3-age-days-anchor.dax	ageing measures case age, not run recency
4-kpi-reconciliation.dax	Exception Rate 8.60%, and the three competing grains
5-status-vocabulary.dax	the exact strings your status measures match on
6-trace-notes-join.dax	trace notes actually land
7-classification-coverage.dax	nothing falls onto the dimension blank member
8-dedupe-recency.dax	the queue shows current state, not first-known state
9-sources-and-contacts.dax	the roster loaded and all five sources are current

Reference documents in the repo

File	Purpose
CURRENT.md	the entry point - what the deliverable is and where to start
NEW-BUILD-FIXES.md	this document, in markdown
WRITEBACK-AND-MAILMERGE.md	List schema, Power Apps formulas, both flow definitions, the blocked-tenant fallback
PBIT-GUIDE.md	Appendix A, in markdown
MEASURE-SWEEP.md	all 25 measures audited, rate-grain reasoning, SharePoint vs API sourcing
ACCOUNT-DRILLDOWN.md	why the account derives from the tracking number; the A25T52 decision

File	Purpose
BUILD_NOTES.md	layout and theme rationale from the first pass
UPS_Acct_Info_.xlsx	24 accounts, A25T52 added, table range already extended to A1:L25

> **Do not run the v5 scripts and the v2 scripts**
>
> tools/model-updates.tmdl and tools/model-fixes.pq are the earlier generation, kept for the reasoning behind them. model-updates-v2.tmdl and model-fixes-v2.pq supersede them entirely. Running both would apply the same changes twice and, in the case of the account key columns, collide.

Still open after all of this

	Why it can wait
Write-back and mail merge	the biggest remaining build - Section 4 and WRITEBACK-AND-MAILMERGE.md
Exception grain decision - 8,724 keys vs 29,957 Is Issue rows vs 46,594 rows	needs a business answer, not a build
Confirm the resolved / escalated literals against production	needs live data flowing
The one trace row whose FA # is the literal string "0"	data entry on the trace sheet; it can never resolve to a contact
Trim QV_Manifest, drop Res_Tracker	performance, not correctness
Folder ingest and dedupe	only once the feed is automated

One to watch, not fix: Resolved Exceptions and Escalated Exceptions return **0** today because nothing in the extract is resolved. That is correct behaviour on this data - but it is indistinguishable from a literal mismatch, which is why Step 10's vocabulary check matters the moment real resolutions start flowing.